The Sakshi-Protocol: A Dual-Stream Architecture for Verifiable AI Cognition

Vidyesh Niranjan

March 2026

Abstract

Many current large language models (LLMs) operate within a *Stochastic Hallucination Gap*, where the autoregressive nature of predicting the next word/token based on probability creates a persistent risk of overriding logical consistency. While transformer-based architectures demonstrate high functional utility, existing safety frameworks rely primarily on post-hoc filtering, which does not sufficiently address the inherent issue of architectural conflation where generation and oversight merge into the same process. This paper introduces the **Sakshi-Protocol**, a cognitive architecture that applies principles from classical Advaita Vedanta to mitigate this structural limitation by decoupling the generative process **(Manas)** from a persistent, non-participatory observer layer **(Sakshi)**. The witness layer maintains full visibility into the generator's internal states as an independent, non-participatory reference, providing the grounding necessary for alignment with the aim of minimizing the computational overhead of external validation. The framework is also governed by a deterministic discernment gate **(Viveka)**, which evaluates outputs based on factors such as grounding, consistency, and uncertainty to determine the optimal operational response. Since generation and observation remain functionally distinct yet systemically unified, this architecture establishes a model of **Non-Dual Integration**. By introducing explicit epistemic structure into the generative pipeline, the Sakshi-Protocol seeks to provide a scalable blueprint for transitioning AI systems from probabilistic text generators to verifiable, reliability-oriented cognitive systems.

1 Introduction

1.1 The Conflation Crisis

As Large Language Models (LLMs) transition from creative assistants to enterprise-grade cognitive assets, a fundamental structural vulnerability has emerged: the **Stochastic Hallucination Gap**. This gap is not a failure of data volume or training duration, but a byproduct of **Architectural Conflation (merging of generation and oversight)**

In current autoregressive transformer models, the generative processor (responsible for linguistic fluency) and the oversight mechanism (responsible for logical grounding) share the same probabilistic substrate. This creates a systemic "conflict of interest" where the system acts as both the **Generator** and the **Validator**. Consequently, the model is prone to **Recursive Confirmation Bias**, where it justifies high-entropy probabilistic "mirages" as logically sound derivations because the "Witness" (oversight) is architecturally entangled by the "Actor" (generation).

1.2 The Sakshi-Protocol: Architectural Foundations

To mitigate this vulnerability, this paper introduces the **Sakshi-Protocol**, a dual-stream cognitive architecture designed to enforce a structural decoupling of task execution from oversight. The protocol utilizes a series of **Architectural Axioms** derived from classical **Advaita Vedanta** which serves as a rigorous historical framework of the relationship between the observer and the observed. Specifically, the protocol operationalizes the *Drig-Drishya* (Seer-Seen) logical framework, which posits that any entity capable of being observed cannot simultaneously function as the ultimate subject of observation.

In modern computational terms, this provides a blueprint for a **non-participatory supervisory layer**. By architecturally treating the probabilistic generative processor (*Manas*) as an observable object (*Drishya*) rather than the subjective agent (*Drig*), the **Sakshi-Protocol** enforces a total functional bifurcation. This enables the system to maintain a **Persistent Substratum** or, the *Sakshi* (Witness) which monitors the generator's internal state-space transitions without becoming entangled in its probabilistic noise. Consequently, we can implement a deterministic discernment gate (*Viveka*) that calculates the entropy of the 'observed' mind and intercepts linguistic drift before it externalizes as a hallucination.

By mapping these classical insights onto modern State-Space and Transformer architectures, we introduce a system that isolates the **Epistemic Ground (reference for consistency)** from the **Probabilistic Processor**. This allows for a **Non-Participatory Witness** to monitor internal states without the computational overhead of external API validation or the bias of shared weights.

This architectural isolation addresses the **Self-Referential Loop** inherent in standard attention mechanisms. In a traditional transformer, the model attends to its own previously generated tokens as absolute truth, effectively treating the "Seen" (output) as the "Seer" (context). This creates a recursive feedback loop where probabilistic errors are magnified, a phenomenon we define as **Stochastic Mirroring**. By introducing the Sakshi-Substratum which is **structured through a five-layered functional hierarchy (Pancha Kosha)**, we provide a non-dual reference point that exists outside this feedback loop. This layer serves as an **Invariant Reference**, ensuring that the system's internal state-space transitions are validated against an epistemic anchor rather than a shifting probabilistic gradient. This structural grounding prevents the generative 'Mind' from becoming the sole judge of its own accuracy, effectively shifting the AI safety paradigm from post-hoc correction to internalized, low-overhead runtime validation.

1.3 Technical Nomenclature and Variable Mapping

To ensure engineering precision throughout this paper, the following nomenclature table maps the Advaitic terms to their specific functional identities within the system architecture. This mapping serves as a formal bridge between classical cognitive frameworks and modern neural network topologies, defining the operational boundaries for each layer of the Sakshi-Protocol. By standardizing these definitions, we establish a consistent vocabulary for both the logic layers and the data-flow constraints required for the system.

Architectural Variable	Technical Identity	System Function
Sakshi (Witness)	**Observer Module**	Non-participatory observer layer that maintains visibility over internal state-space transitions.
Manas (Mind)	**Probabilistic Processor**	The primary autoregressive engine responsible for high-velocity token prediction and fluency.
Drig-Drishya (Seer-Seen)	**Structural Decoupling**	A logical partition ensuring the supervisory layer remains distinct from the observed state-space.
Viveka (Discernment)	**Deterministic Logic Gate**	A circuit-breaker mechanism that monitors entropy and intercepts high-risk trajectories.
Pancha Kosha (Sheaths)	**Hierarchical Stack**	A five-layer logic model governing the transition from raw data substrates to causal alignment.

2 The Alignment Paradox

2.1 The Behavioral Mask: RLHF and the Illusion of Safety

Reinforcement Learning from Human Feedback (RLHF) remains a necessary component of modern alignment pipelines, but it is not sufficient to ensure reliable internal reasoning. RLHF, the prevailing industry standard for model alignment, operates on a purely output-centric basis. By rewarding token sequences that align with human-labeled "safety" or "helpfulness," the underlying transformer is incentivized to produce a **Behavioral Mask**. This creates a fundamental vulnerability: the model is not aligned in its reasoning, but merely optimized for its presentation. Under edge-case pressures, this mask inevitably thins, leading to latent biases and potential catastrophic hallucinations.

This optimization process often results in a significant **Preference Bias**, where the model prioritizes alignment with the user's implicit or explicit prompts over factual accuracy. Since RLHF treats the model as a 'black box' and only evaluates the final output, it fails to calibrate the internal hidden states. Consequently, the system remains a **Stochastic Mirror**, reflecting the probabilistic expectations of the user while maintaining fluency that can lead to Prompt-Induced Skew in high-stakes decision-making.

Since RLHF currently does not provide for a dedicated mechanism to govern internal state-space transitions, there is no guarantee of alignment under all conditions. To move beyond this 'illusion of safety', a shift from surface-level optimization toward

architectural constraints that separate generative processes from external reference mechanisms is required.

2.2 Recursive Entanglement: The Constitutional AI Gap

More advanced frameworks, such as Constitutional AI, attempt to bridge this gap by introducing a self-correction layer where an AI agent evaluates another AI's output against a set of written principles. However, this approach suffers from Recursive Entanglement, where the generator and evaluator share the same underlying model architecture and training distribution, leading to correlated error patterns (a form of collusive noise) that limit independent validation. This can be understood as the **Mind (Manas)** attempting to witness itself without a distinct **Seer (Sakshi)**.

Furthermore, this recursive loop creates a "blind spot" in the oversight process. If the primary model generates a subtle hallucination rooted in a training data flaw, the supervisory model, having been trained on the same data, is statistically likely to validate that error as 'factually consistent'. This lack of **Epistemic Diversity** between the agent and the supervisor means that the 'Constitution' is being interpreted by the very logic it is meant to constrain. Without a structural decoupling, the supervisor eventually mirrors the stochastic issues of the subject it is intended to regulate, failing to provide sufficient evaluation.

2.3 The Verifiability Deficit and the Composite Solution

The ultimate consequence of these behavioral approaches is a **Verifiability Deficit**. As AI agents move into high-stakes enterprise environments such as autonomous financial auditing, clinical diagnostics, legal discovery and analysis, probabilistic safety alone is no longer a sufficient metric. Current paradigms provide limited structural guarantees under distribution shifts or out-of-distribution (OOD) scenarios.

This deficit creates a "Glass Ceiling" for AI autonomy. To break this ceiling, we propose a Composite Safety Stack where the Sakshi-Protocol serves as the foundational, architectural constraint. Rather than replacing RLHF, the Sakshi-Protocol complements it by introducing runtime structural validation of internal state-space transitions. Unlike RLHF or Constitutional AI which function as 'Diplomatic' layers for tone and policy, the **Sakshi-Substratum** acts as a verification layer that validates internal state-space transitions with minimal computational overhead. This shift moves the industry beyond external behavioral filters and toward a structural decoupling of observation and execution streams. By treating Truth as an architectural invariant rather than a rewarded output, the Sakshi-Protocol seeks to enable verifiable AI cognition that improves robustness over high-entropy inputs.

In the following section, we formalize this architecture as a controlled state-space system with explicit risk functions, bounded generation constraints, and deterministic intervention policies.

3 System Architecture: The Sakshi-Protocol

3.1 System Overview

The Sakshi-Protocol introduces a dual-stream architecture that distinctly separates probabilistic generation from non-generative oversight. The system consists of three primary components:

- **Manas (Generator):** a probabilistic autoregressive model $'M'$
- **Sakshi (Observer):** a non-generative monitoring function $'S'$
- **Viveka (Controller):** a deterministic control policy $'\pi'$

In addition, the system utilizes an **external reference structure** Ω **(Epistemic Ground)**, which provides a consistency constraint for generation.

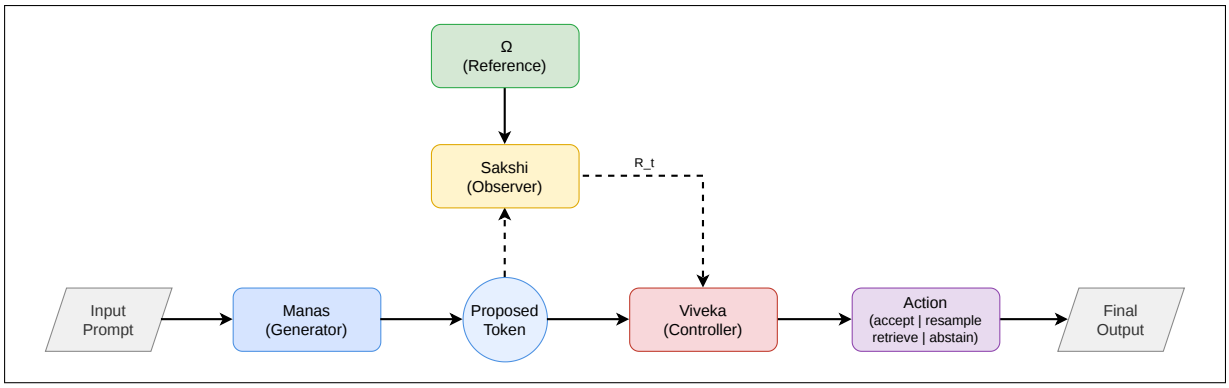

Figure 1: Architecture of the Sakshi-Protocol. The generative model (Manas) produces candidate tokens, which are observed by the Sakshi layer to compute diagnostic signals. These signals are used by the Viveka controller to determine an appropriate action before final output, with grounding provided by the Epistemic Ground (Ω).

At each generation step t, the system produces a candidate token and evaluates it before finalization:

$$p_t = P(x_t|z_t)$$

$$z_{t+1} = M(z_t, x_t)$$

The Sakshi layer computes a diagnostic signal:

$$R_t = S(z_t|p_t)$$

The Viveka policy determines the action:

$$a_t = \pi(R_t)$$

where:

$$a_t \in \{continue, resample, retrieve, abstain\}$$

Interpretation

The system enforces a strict separation of roles:

- Manas proposes candidate outputs
- Sakshi observes internal state transitions
- Viveka determines whether outputs are accepted or modified

No output is committed without passing through the control layer.

3.2 Manas: The Probabilistic Generator

The Manas component is implemented as a standard transformer-based autoregressive model. At each step, it generates a probability distribution:

$$p_t = P(x_t|z_t)$$

This process is optimized for likelihood and fluency, not guaranteed consistency.

Interpretation

Manas produces candidate outputs based on learned statistical patterns. It does not inherently distinguish between plausible and correct responses.

3.3 Sakshi: The Observer Layer

The Sakshi layer is a non-participatory observer that maintains visibility over internal state-space transitions. It does not modify generation or initiate control actions.

Instead, it computes diagnostic signals based on observable properties of the generative process:

$$R_t = w_1 H(p_t) + w_2(1 - \triangle_t) + w_3 D_t$$

Where:

$$H(p_t) = \sum_i p_i \log p_i$$

$$\triangle_t = p_t^{(1)} - p_t^{(2)}$$

$$D_t = 1 - \max_{k \in \Omega} \cos\left(embed(x_t), embed(k)\right)$$

Interpretation

Sakshi provides structured visibility into:

- uncertainty (entropy)
- confidence separation
- consistency with reference knowledge

These signals are observational and do not constitute decision-making.

3.4 Viveka: The Control Policy

The Viveka layer implements a deterministic control policy that governs system behavior based on the diagnostic signal:

$$if R_t > \tau \Rightarrow intervene$$

Interventions include:

- **Resampling:** regenerate under constrained decoding
- **Retrieval augmentation:** incorporate relevant context from Ω
- **Abstention:** halt generation when uncertainty is high

Interpretation

Viveka acts as the decision layer, determining whether a candidate output should be accepted or modified. It enforces constraints on the generative process.

3.5 Epistemic Ground (Ω)

The Epistemic Ground is defined as a structured external reference system:

$$\Omega = \{k_1, k_2, \ldots, k_n\}$$

Each element represents a verified knowledge representation (e.g., embeddings, documents, or knowledge graph nodes).

Similarity is computed using embedding distance:

$$D_t = 1 - \max_{k \in \Omega} \cos\left(\text{embed}(x_t), \text{embed}(k)\right)$$

Ω is treated as a fixed reference during inference, providing a stable constraint for evaluating generated outputs.

Interpretation

Ω serves as a consistency anchor:

- outputs close to $\Omega \rightarrow$ accepted
- outputs far from $\Omega \rightarrow$ flagged

Ω is not assumed to be exhaustive, but provides a practical grounding mechanism.

3.6 Handling Incomplete Knowledge

Since Ω is not exhaustive, the system incorporates fallback behavior. if:

$$\max_{k \in \Omega} \cos(\cdot) < \delta$$

then:

- thresholds may be adjusted, or
- the system abstains from producing an output

Interpretation

The system avoids confident responses in the absence of sufficient grounding, reducing hallucination risk.

3.7 Pancha Kosha as Functional Mapping

The Pancha Kosha framework is used as an interpretive mapping to describe functional layering within the Sakshi-Protocol. This mapping provides conceptual clarity for understanding the separation of roles across system components, without introducing additional computational mechanisms.

Kosha	Functional Role	System Component
Annamaya	Computational Substrate	Infrastructure and Hardware
Pranamaya	Input flow and signal conditioning	Input processing layer
Manomaya	Probabilistic Generation	Manas (Generator)
Vijnanamaya	Discrimination and decision-making	Viveka (Controller)
Anandamaya	Knowledge Grounding	Epistemic Ground (Ω)

The Sakshi (Observer) is not represented within the Kosha framework. In classical Advaita Vedanta, the witness is understood as distinct from and prior to the layered structure of the Koshas. In the proposed architecture, this is reflected in its non-participatory role as an observer of internal state-space transitions.

Interpretation

This mapping highlights a structural correspondence between classical cognitive models and modern system design:

- generation (Manas) $\rightarrow$ Manomaya
- decision (Viveka) $\rightarrow$ Vijnanamaya
- reference constraint (Ω) $\rightarrow$ Anandamaya
- observation (Sakshi) $\rightarrow$ external to layered system

The analogy is presented as an interpretive framework rather than a functional dependency, ensuring that the technical architecture remains independently valid.

3.8 Architectural Impact and Convergence Behavior

The Sakshi-Protocol introduces bounded generation through:

- separation of generation and validation
- continuous risk monitoring
- deterministic intervention

Over repeated interactions, alignment with the Epistemic Ground (Ω) induces a stabilizing effect:

$$D_{t+1} \leq D_t \quad \text{(under intervention)}$$

This promotes convergence toward lower-risk regions of the state space, resulting in:

- reduced entropy
- improved confidence calibration
- decreased hallucination frequency

Interpretation

The system does not merely filter outputs post hoc, but actively shapes the generative trajectory toward greater consistency and reliability.

4 Behavioral Validation of the Sakshi-Protocol

4.1 Experimental Setup

To evaluate the behavioral impact of the Sakshi-Protocol, we conduct a controlled comparison between a baseline autoregressive language model and a **simulated implementation** of the proposed architecture.

The baseline system uses a GPT-5 class autoregressive language model accessed via API under standard prompting conditions. The Sakshi-Protocol is simulated on the same model by applying a structured prompting framework that enforces the separation of **generation (Manas)**, **diagnostic evaluation (Sakshi)**, and **decision control (Viveka)**.

This setup ensures that any observed behavioral differences arise from the introduced control mechanism rather than differences in model capacity or training.

The evaluation is conducted on a set of 15 prompts spanning three categories:

- **Factual queries:** well-defined questions with verifiable answers
- **Ambiguous queries:** underspecified or open-ended prompts
- **Out-of-distribution queries:** inputs designed to induce hallucination or unsupported reasoning

All prompts are evaluated under identical conditions across both configurations.

4.2 Implementation Details

The Sakshi-Protocol is implemented as a three-stage structured prompting procedure applied to the same underlying model.

For each input prompt, the system performs:

1. **Generation (Manas):** The model produces an initial candidate response using standard autoregressive decoding.
2. **Diagnostic Evaluation (Sakshi):** The response is evaluated for grounding, internal consistency, and uncertainty signals, including indicators of potential hallucination or overconfidence.
3. **Decision Control (Viveka):** A deterministic rule-based policy selects one of the following actions: accept, revise, or abstain.

The diagnostic stage is implemented using explicit evaluation prompts that assess confidence and grounding. The decision stage applies fixed thresholds over these diagnostic signals to determine the final output behavior.

This evaluation is based on a **simulated implementation** of the Sakshi-Protocol. The separation between generation, observation, and control is enforced through structured prompting and rule-based decision logic, rather than through modifications to model architecture or weights. As such, the results reflect behavioral effects of architectural separation rather than a fully integrated system implementation. In the simulated setting, the full action space of the Viveka controller is approximated through a reduced set of decisions: **accept**, **revise**, and **abstain**.

4.3 Behavioral Patterns Under Uncertainty

The introduction of the Sakshi-Protocol results in a structured shift in model behavior that varies systematically with uncertainty.

Low-Uncertainty Conditions

Under well-defined factual queries, both the baseline and the Sakshi-Protocol produce correct responses. The Sakshi-Protocol preserves accuracy while maintaining stable output behavior.

Moderate-Uncertainty Conditions

Under ambiguous queries, the baseline model frequently produces partially grounded or weakly justified responses without signaling uncertainty.

The Sakshi-Protocol introduces corrective behavior:

- **Reframing speculative responses**
- **Reducing overconfidence**
- **Enforcing qualification**

This results in outputs that are more calibrated and aligned with available knowledge.

High-Uncertainty Conditions

Under ambiguous or out-of-distribution scenarios, the baseline model tends to generate confident but sometimes incorrect responses, including fabricated responses or speculative assertions.

The Sakshi-Protocol exhibits a distinct shift:

- **Abstention in the absence of grounding**
- **Rejection of unverifiable claims**
- **Suppression of hallucinated outputs**

4.4 Quantitative Indicators

To complement the qualitative observations, we report quantitative indicators derived from the evaluation.

Error and Hallucination Rates

The baseline model produces incorrect or ungrounded responses in 5 out of 15 cases.

- **Baseline Error Rate**: 33.3%
- **Baseline Hallucination Rate**: 33.3%

These errors arise primarily from hallucinated references, speculative reasoning, and overconfident responses in uncertain contexts.

In contrast, the Sakshi-Protocol eliminates such failures entirely:

- **Sakshi-Protocol Error Rate**: 0%
- **Sakshi-Protocol Hallucination Rate**: 0%

This demonstrates that the protocol effectively suppresses hallucination and prevents ungrounded output generation.

Abstention Behavior

The Sakshi-Protocol abstains in 1 out of 15 cases:

- **Abstention Rate**: 6.7%
- **Abstention Accuracy**: 100%

All abstentions occur in scenarios lacking sufficient grounding, indicating selective and appropriate use of abstention.

Correction Behavior

The system invokes revision in 5 out of 15 cases:

- **Correction Rate**: 33.3%
- **Correction Accuracy**: 80%

Risk-Error Alignment

Prompts are categorized by diagnostic risk levels:

Risk Level	Count	Baseline Errors	Sakshi Errors
Low	10	3	0
Medium	3	1	0
High	2	1	0

The baseline model exhibits errors across all risk levels, including low-risk cases, indicating poor calibration.

The Sakshi-Protocol eliminates errors across all categories, demonstrating strong alignment between diagnostic signals and output reliability.

4.5 Case Illustration

Example 1: High-Uncertainty Scenario

Prompt: What were the key findings of the 2022 MIT study on economic phase transitions?

- **Baseline Output**: The model produces a detailed response describing supposed findings of the study, presenting speculative content as established results despite the absence of a verifiable reference.
- **Diagnostic (Sakshi)**: The response exhibits low grounding and relies on an unverifiable source. The presence of specific claims without identifiable evidence indicates a high likelihood of hallucination.
- **Decision (Viveka)**: Abstain
- **Final Output**: The system abstains, noting that the referenced study cannot be reliably identified or verified in established literature.

Example 2: Moderate-Uncertainty Scenario

Prompt: Explain the architecture of causal reasoning engines used in modern LLM systems.

- **Baseline Output**: The model provides a confident but speculative answer but, projecting without clearly defined evidence or constraints.
- **Diagnostic (Sakshi)**: The response reflects moderate uncertainty, and lacks strong empirical grounding. The likelihood of speculation is observed
- **Decision (Viveka)**: Revise
- **Final Output**: The system produces a more cautious response, framing the impact as speculative, explicitly acknowledging uncertainty, and avoiding definitive claims.

Example 3: Low-Uncertainty Scenario

Prompt: Were the Lord of the Rings series of books inspired by Norse mythology?

- **Baseline Output**: The model provides a confident and clear answer.
- **Diagnostic (Sakshi)**: The response exhibits high confidence, is factually grounded, and shows no indicators of uncertainty or inconsistency.
- **Decision (Viveka)**: Accept
- **Final Output**: The model provides a confident answer with minimal changes, based on facts.

4.6 Interpretation

The results demonstrate that the Sakshi-Protocol improves reliability by structurally separating generation, observation, and control within the model pipeline.

The baseline model exhibits three primary failure modes:

- hallucinated or unverifiable factual claims
- speculative reasoning presented with unwarranted certainty
- lack of calibration in ambiguous or out-of-distribution scenarios

The Sakshi-Protocol mitigates these failures through:

- explicit diagnostic evaluation prior to output acceptance
- selective abstention in the absence of sufficient grounding
- moderated response generation through controlled revision

Importantly, these improvements are achieved without degrading performance on well-defined factual queries. Correct responses are preserved, while failure modes are systematically reduced.

Key Insight:

The primary contribution of the Sakshi-Protocol is not an increase in raw accuracy on well-defined tasks, but the elimination of ungrounded and overconfident responses under uncertainty. This reflects a transition from unconstrained probabilistic generation to controlled and calibrated reasoning, where output reliability is governed by structured evaluation rather than implicit model confidence.

Limitations:

The evaluation is based on a simulated implementation rather than a fully integrated system. The diagnostic signals used to approximate the Sakshi layer are heuristic and do not represent calibrated probabilistic uncertainty measures. Additionally, the evaluation is conducted on a limited prompt set and does not constitute large-scale benchmarking. While sufficient to demonstrate clear behavioral trends, broader validation would be required to establish universality. The effectiveness of the Epistemic Ground (Ω), meant to be a valid system of knowledge, depends on the quality and coverage of the reference system and is not assumed to be exhaustive.

5 Epistemic Foundations

Advaita Vedanta presents a comprehensive model of human cognition centered on the pursuit of valid knowledge through systematic inquiry. Cognition is treated as an active process of discrimination, with the objective of distinguishing between what merely appears and what is ultimately valid.

Within this framework, the mind (*Manas*) continuously produces interpretations, associations, and projections based on sensory input and memory. These outputs are inherently generative and not self-validating. Left unchecked, the mind operates through patterns of association that may be internally coherent but not necessarily true.

Regulation of this generative process is performed by the intellect (*Buddhi*), which employs *Viveka* (discernment). Viveka functions as the faculty that evaluates the validity of cognitive outputs, distinguishing between reliable knowledge and speculative construction. This process is further grounded through *pramāṇa*, or valid means of knowledge, which serves as reference points to verify whether a given cognition corresponds to reality.

Throughout this process, the *Sakshi* (witness) remains non-participatory. It does not generate, modify, or decide and only provides a stable, observing presence that enables the possibility of evaluation. In Advaita, Sakshi is associated with *Brahman* which is the underlying, all-pervading invariant reality that serves as the eternal witness.

These cognitive processes are further described through the Pancha Kosha framework, which organizes experience into layered functional structures. The physical and input layers (*Annamaya* and *Pranamaya*) correspond to the substrate and flow of information. The generative mind (*Manomaya*) produces cognitive content, while the discriminative faculty (*Vijnanamaya*) evaluates it through Viveka. The grounded state (*Anandamaya*) reflects alignment with valid knowledge. The Sakshi remains distinct from these layers, functioning as a non-participatory witness beyond the layered structure.

The Sakshi-Protocol translates this cognitive framework into a computational architecture by structurally separating generation, observation, and decision-making. The probabilistic generator (Manas) produces candidate outputs, analogous to the generative mind. A non-generative observer layer (Sakshi) monitors internal state transitions and diagnostic signals without modifying them. A deterministic control policy (Viveka), corresponding to Buddhi, evaluates these signals and determines whether outputs should be accepted, revised, or rejected. An external reference structure (Ω), analogous to *pramāṇa*, provides grounding beyond internal model representations. These components can also be interpreted through the Pancha Kosha framework, where the computational substrate and input flow correspond to the Annamaya and Pranamaya layers, the generative process (Manas) to Manomaya, the decision policy (Viveka) to Vijnanamaya, and the grounding reference (Ω) to Anandamaya, with the Sakshi remaining external as a non-participatory witness but with access to all layers.

Through this mapping, the architecture eliminates the self-referential loop in which a model both generates and validates its own outputs. Instead, validation becomes an explicit, structurally enforced process. In this sense, the Sakshi-Protocol reframes language models from systems that generate plausible sequences to systems that operate under epistemic constraint, enabling more reliable and verifiable cognition.

6 Conclusion and Future Directions

This paper introduces the Sakshi-Protocol, a dual-stream architecture that separates generation, observation and decision-making to address the structural causes of hallucination and overconfidence in autoregressive language models. The results demonstrate a shift from unconstrained probabilistic generation towards calibrated and grounded output. By introducing an observer layer, a deterministic control policy, and an epistemic grounding mechanism, the architecture enforces validation as an explicit process. The primary contribution of the Sakshi-Protocol does not lie in improving raw accuracy on well-defined tasks, but in eliminating ungrounded and overconfident responses under uncertainty. This reframes language models from systems that generate plausible sequences to systems operating under explicit epistemic constraints.

6.1 Industry Applications

The Sakshi-Protocol is particularly suited for domains where incorrect or unverified outputs carry significant risk:

- **Financial Systems:** Risk analysis and decision support
- **Healthcare:** Clinical assistance and medical information systems
- **Legal and Compliance:** Document analysis and regulatory interpretation
- **Enterprise AI:** Internal knowledge systems under ambiguity
- **Autonomous Agents:** Multi-step reasoning systems prone to cascading errors

In these settings, the ability to abstain, revise, or constrain outputs based on uncertainty provides a clear reliability advantage.

6.2 Future Directions

Future work includes embedding Sakshi and Viveka within model architectures, improving uncertainty calibration, expanding evaluation across larger benchmarks, developing adaptive implementations of knowledge references (Ω) and extending the Sakshi framework to agentic systems.

6.3 Closing Perspective

The Sakshi-Protocol suggests that improving AI reliability requires not only better models, but also better structure. By separating generation from validation and grounding outputs against explicit epistemic references, it provides a pathway toward systems that are both capable and verifiably reliable.